

MAX HEAP SORT

```

import time

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def max_heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    element = int(input("Element " + str(i + 1) + ": "))
    arr.append(element)

print("\nOriginal Array:", arr)

start_time = time.perf_counter()

max_heap_sort(arr)

end_time = time.perf_counter()

execution_time = end_time - start_time

print("Sorted Array:", arr)
print("Execution Time:", execution_time, "seconds")

print("\nTime Complexity:")
print("Best Case    : O(n log n)")
print("Average Case : O(n log n)")
print("Worst Case    : O(n log n)")

```

```

Enter the number of elements: 6
Enter the elements:
Element 1: 4
Element 2: 7
Element 3: 2
Element 4: 8
Element 5: 5
Element 6: 1

Original Array: [4, 7, 2, 8, 5, 1]
Sorted Array: [1, 2, 4, 5, 7, 8]
Execution Time: 9.029400001736576e-05 seconds

Time Complexity:
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case    : O(n log n)

```